JAVASCRIPT

Functions • Arrays • Array Methods • Modules

Beginner-to-practical guide | Hinglish explanations + runnable code

Is guide mein kya seekhoge?

- Functions: declaration, expression, arrow, parameters, return, scope, callbacks, higher-order functions
- Arrays: creation, indexing, updating, looping, copying, destructuring, spread/rest
- Array methods: map, filter, reduce, forEach, find, some, every, sort, slice, splice, and more
- Modules: export, import, default export, named export, and practical file structure
- Common mistakes, mini projects, practice questions, and quick revision sheet

Learning tip: Har code example ko khud type karo, run karo, phir values badal kar output predict karo.

1. JavaScript Functions — foundation

Function reusable code ka block hota hai. Ek baar function likho aur zaroorat padne par use call karke chalao.

1.1 Function declaration

```
function greet() {
    console.log("Hello!");
}

greet(); // function call
```

Samjho: `function` keyword function banata hai; `greet` uska naam hai; `()` call karta hai; `{ }` body hai.

1.2 Parameters and arguments

```
function add(a, b) {  // a, b = parameters
    return a + b;
}

let total = add(8, 9); // 8, 9 = arguments
console.log(total);   // 17
```

Parameter function definition mein placeholder hota hai. Argument call karte waqt di gayi actual value hoti hai.

1.3 return vs console.log

```
function square(n) {
    return n * n;
}

let answer = square(5);
console.log(answer); // 25
```

return value ko function ke bahar bhejta hai. console.log() sirf console par dikhata hai. Dono same kaam nahi karte.

1.4 Function expression

```
const multiply = function(a, b) {
    return a * b;
};

console.log(multiply(4, 3)); // 12
```

1.5 Arrow function

```
const subtract = (a, b) => {
    return a - b;
};

// Short form: implicit return
const double = n => n * 2;

console.log(subtract(9, 4)); // 5
console.log(double(6));       // 12
```

Arrow function mein => arrow syntax hai. Single parameter par parentheses optional hain. Ek expression ko braces ke bina likhne par value automatically return hoti hai.

1.6 Default parameters

```
function welcome(name = "friend") {
    return `Hello, ${name}!`;
}

console.log(welcome());       // Hello, friend!
```

```
console.log(welcome("Aman")); // Hello, Aman!
```

1.7 Rest parameters

```
function sumAll(...numbers) {
    return numbers.reduce((total, n) => total + n, 0);
}

console.log(sumAll(2, 3, 4)); // 9
```

`...numbers` multiple arguments ko ek array mein collect karta hai. Rest parameter function ke parameter list mein last hona chahiye.

1.8 Scope: local and global

```
let siteName = "My App"; // global/module scope

function showName() {
    let message = "Welcome"; // local to function
    console.log(siteName, message);
}

showName();
// console.log(message); // ReferenceError
```

Function ke andar declared `let`/`const` variable normally function ke bahar accessible nahi hota.

2. Callback and Higher-Order Functions

Callback function woh function hai jo kisi doosre function ko argument ke roop mein pass kiya jata hai.

```
function greet(name) {
    console.log("Hello " + name);
}

function processUser(callback) {
    callback("Aman");
}

processUser(greet); // Hello Aman
```

Yahan `greet` callback hai. `processUser` us callback ko receive karke call karta hai.

2.1 Anonymous callback

```
setTimeout(() => {
    console.log("2 seconds later");
}, 2000);
```

Callback ko naam wala ya anonymous function dono tarah pass kar sakte ho.

2.2 Higher-order function (HOF)

Jo function doosre function ko argument leta hai, ya function return karta hai, woh higher-order function kehlata hai. Array ke `map()`, `filter()` aur `reduce()` common examples hain.

2.3 Callback in map

```
const nums = [1, 2, 3];

const doubled = nums.map((value, index, array) => {
    return value * 2;
});

console.log(doubled); // [2, 4, 6]
```

Important: map callback arguments ka order `(element, index, array)` hota hai. Parameter ke naam tum khud rakhte ho; naam badalne se unka role nahi badalta.

3. Arrays — basics

Array ek ordered collection hai jisme multiple values ek variable mein store hoti hain. Index 0 se start hota hai.

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits[0]);       // apple
console.log(fruits.length);   // 3
fruits[1] = "orange";         // update
fruits.push("grapes");        // add at end
```

3.1 Create arrays

```
const empty = [];
const scores = [80, 92, 75];
const mixed = ["Aman", 21, true];
```

3.2 Add and remove manually

```
const items = ["pen", "book"];

items.push("bag");     // end mein add
items.pop();           // end se remove
items.unshift("pencil"); // start mein add
items.shift();         // start se remove

console.log(items);
```

3.3 Check array and access safely

```
const data = [10, 20, 30];

console.log(Array.isArray(data)); // true
console.log(data.at(-1));         // 30 (last item)
console.log(data[99]);            // undefined
```

3.4 Loop through arrays

```
const names = ["Aman", "Riya", "Kabir"];

for (let i = 0; i < names.length; i++) {
    console.log(names[i]);
}

for (const name of names) {
    console.log(name);
}

names.forEach((name, index) => {
    console.log(index, name);
});
```

`for...of` values ke liye useful hai. Classic `for` loop index aur control deta hai. `forEach` har item par callback chalata hai.

3.5 Copying arrays

```
const original = [1, 2, 3];

const copy1 = [...original];       // spread copy
const copy2 = original.slice();    // shallow copy

copy1.push(4);
console.log(original); // [1, 2, 3]
```

Spread aur slice yahan shallow copy banate hain. Nested objects/arrays ke andar ki values shared reh sakti hain.

3.6 Destructuring

```
const colors = ["red", "green", "blue"];

const [first, second, ...others] = colors;

console.log(first);  // red
console.log(second); // green
console.log(others); // ["blue"]
```

4. Array methods — add, remove, copy, search

Neeche methods ko groups mein rakha gaya hai. Dhyan do kaun original array ko change karta hai.

Method	Kaam	Original array?
push()	End mein item add	Change karta hai
pop()	End se item remove	Change karta hai
unshift()	Start mein item add	Change karta hai
shift()	Start se item remove	Change karta hai
splice()	Add/remove/replace by index	Change karta hai
slice()	Array ka selected portion copy	Change nahi karta
concat()	Arrays/values jodkar new array	Change nahi karta
includes()	Value present hai?	Change nahi karta
indexOf()	First matching index; na mile -1	Change nahi karta
join()	Array ko string mein convert	Change nahi karta
reverse()	Order ulta	Change karta hai
sort()	Items sort	Change karta hai

4.1 push, pop, shift, unshift

```
const a = [2, 3];
a.push(4);       // [2, 3, 4]
a.pop();         // [2, 3]
a.unshift(1);    // [1, 2, 3]
a.shift();       // [2, 3]
```

4.2 splice vs slice

```
const nums = [10, 20, 30, 40, 50];

const part = nums.slice(1, 3);
console.log(part); // [20, 30]
console.log(nums); // original unchanged

nums.splice(1, 2, 99);
console.log(nums); // [10, 99, 40, 50]
```

slice(start, end) end index ko include nahi karta aur original ko nahi badalta. splice(start, deleteCount, ...items) original array ko modify karta hai.

4.3 sort numbers correctly

```
const values = [100, 5, 20, 3];

values.sort((a, b) => a - b);
console.log(values); // [3, 5, 20, 100]

values.sort((a, b) => b - a);
console.log(values); // [100, 20, 5, 3]
```

Default sort values ko strings ki tarah compare karta hai. Numeric sorting ke liye comparator use karo. a - b ascending; b - a descending.

5. map(), filter(), reduce() — must know

5.1 map(): transform every element

```
const nums = [1, 2, 3];

const doubled = nums.map((value) => {
    return value * 2;
});

console.log(doubled); // [2, 4, 6]
```

map har element par callback chalata hai aur callback ke returned values se NEW array banata hai. Original array unchanged rehta hai (jab tak callback khud usse mutate na kare).

5.2 map callback arguments

```
const letters = ["a", "b", "c"];

const result = letters.map((element, index, array) => {
    return `${index}: ${element}`;
});

console.log(result);
// ["0: a", "1: b", "2: c"]
```

Map callback signature: `(element, index, array)`. Agar tum `(acc, val)` likhoge, to pehla parameter phir bhi element hi hoga; naam se accumulator nahi ban jata.

5.3 filter(): select matching elements

```
const nums = [5, 12, 18, 21, 30];

const big = nums.filter((value) => {
    return value >= 18;
});

console.log(big); // [18, 21, 30]
```

filter callback ka truthy/falsy result dekhta hai. true hone par original element new array mein rakhta hai; false hone par skip karta hai.

5.4 reduce(): accumulate into one result

```
const nums = [10, 20, 30];

const total = nums.reduce((acc, value) => {
    return acc + value;
}, 0);

console.log(total); // 60
```

Reduce callback signature: `(accumulator, currentValue, index, array)`. Initial value `0` accumulator ki starting value hai. Har return next iteration ka accumulator ban jata hai.

5.5 Reduce step-by-step

Step	Accumulator	Current value	Return
Start	0	—	0
1	0	10	10
2	10	20	30
3	30	30	60

5.6 map vs filter vs reduce vs forEach

Method	Purpose	Typical result
map()	Transform each item	New array (same length)
filter()	Keep matching items	New array (same or shorter)
reduce()	Accumulate values	Any single/final value
forEach()	Run side effect per item	Always undefined return

6. More important array methods

6.1 find() and findIndex()

```
const users = [
    { id: 1, name: "Aman" },
    { id: 2, name: "Riya" }
];

const user = users.find(u => u.id === 2);
const index = users.findIndex(u => u.id === 2);

console.log(user);  // { id: 2, name: "Riya" }
console.log(index); // 1
```

find first matching element deta hai; findIndex uska index deta hai. Match na mile to respectively undefined aur -1.

6.2 some() and every()

```
const marks = [45, 67, 82];

console.log(marks.some(m => m >= 80));  // true
console.log(marks.every(m => m >= 35)); // true
```

some = kam se kam ek item condition pass kare? every = kya saare items pass karte hain?

6.3 flat() and flatMap()

```
const nested = [1, [2, 3], [4, [5]]];

console.log(nested.flat());    // [1, 2, 3, 4, [5]]
console.log(nested.flat(2));   // [1, 2, 3, 4, 5]

const words = ["hi", "ok"];
console.log(words.flatMap(w => w.split("")));
// ["h", "i", "o", "k"]
```

flat nested arrays ko specified depth tak flatten karta hai. flatMap pehle map karta hai, phir one-level flatten.

6.4 fill() and Array.from()

```
const a = new Array(3).fill(0);
console.log(a); // [0, 0, 0]

const squares = Array.from({ length: 5 }, (_, i) => i * i);
console.log(squares); // [0, 1, 4, 9, 16]
```

6.5 at() and includes()

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits.at(-1));          // mango
console.log(fruits.includes("pear")); // false
```

6.6 toSorted(), toReversed(), toSpliced()

```
const original = [3, 1, 2];

const sorted = original.toSorted((a, b) => a - b);
const reversed = original.toReversed();

console.log(original); // [3, 1, 2] unchanged
console.log(sorted);   // [1, 2, 3]
console.log(reversed); // [2, 1, 3]
```

Modern non-mutating alternatives: toSorted, toReversed, toSpliced. Older browsers/runtimes may need support checks.

7. Objects inside arrays

Real projects mein arrays ke andar objects bahut common hote hain.

```
const students = [
    { name: "Aman", marks: 85 },
    { name: "Riya", marks: 92 },
    { name: "Kabir", marks: 70 }
];

// Names only
const names = students.map(student => student.name);

// Students who scored 80+
const toppers = students.filter(student => student.marks >= 80);

// Total marks
const totalMarks = students.reduce((sum, student) => {
    return sum + student.marks;
}, 0);

console.log(names);
console.log(toppers);
console.log(totalMarks); // 247
```

7.1 Count frequency with reduce

```
const fruits = ["apple", "banana", "apple", "mango", "banana", "apple"];

const counts = fruits.reduce((acc, fruit) => {
    acc[fruit] = (acc[fruit] || 0) + 1;
    return acc;
}, {});

console.log(counts);
// { apple: 3, banana: 2, mango: 1 }
```

Yahan initial accumulator empty object {} hai. Har fruit ki count property update hoti hai.

7.2 Group items

```
const people = [
    { name: "Aman", city: "Bhilai" },
    { name: "Riya", city: "Raipur" },
    { name: "Kabir", city: "Bhilai" }
];

const byCity = people.reduce((groups, person) => {
    const city = person.city;
    if (!groups[city]) groups[city] = [];
    groups[city].push(person);
    return groups;
}, {});
```

8. JavaScript Modules — split code across files

Module ka matlab code ko alag files mein organize karna. Ek file se function/variable export karo, doosri file mein import karo. ES Modules mein `export` aur `import` use hote hain.

8.1 Named export

math.js

```
export const pi = 3.14159;

export function add(a, b) {
    return a + b;
}

export function multiply(a, b) {
    return a * b;
}
```

app.js

```
import { pi, add, multiply } from "./math.js";

console.log(add(2, 3));       // 5
console.log(multiply(3, 4));  // 12
console.log(pi);              // 3.14159
```

Named export ko import karte waqt same exported name curly braces mein likhte hain. Alias bhi de sakte ho: `import { add as sum }`.

8.2 Default export

greet.js

```
export default function greet(name) {
    return `Hello, ${name}`;
}
```

app.js

```
import greet from "./greet.js";

console.log(greet("Aman")); // Hello, Aman
```

Ek module mein ek default export hota hai. Import karte waqt default export ka naam apni marzi se rakh sakte ho.

8.3 Named + default together

```
// tools.js
export const version = "1.0";
export default function start() {
    console.log("Started");
}

// app.js
import start, { version } from "./tools.js";
```

8.4 Browser setup

```
<!-- index.html -->
<script type="module" src="./app.js"></script>
```

Browser mein module script ke liye `type="module"` use karo. Local files ko directly file:// se kholne par module/CORS issues aa sakte hain; local development server use karo.

8.5 Node.js setup

Node.js mein ES Modules ke liye aam taur par package.json mein `"type": "module"` set karo, ya .mjs extension use karo.

```
// package.json
{
  "type": "module"
}
```

CommonJS bhi Node.js mein milta hai: `require()` aur `module.exports`. ES Modules ka syntax `import/export` hai. Dono module systems ko bina samjhe mix mat karo.

9. Common mistakes — inhe avoid karo

map mein parameter ko acc naam dena
Naam kuch bhi ho sakta hai. map ka first argument current element hota hai, accumulator nahi.

map ke andar return bhool jana
Braces { } use kiye hain to explicit return chahiye. Return na ho to result undefined hota hai.

map se filtering karna
Condition se items select karne ke liye filter() use karo.

reduce mein initial value bhoolna
Empty array par initial value ke bina reduce error de sakta hai. Sum ke liye 0, product ke liye 1 do.

sort numbers without comparator
Default sort lexical/string-style hota hai. Numbers ke liye (a, b) => a - b use karo.

const array ko immutable samajhna
const variable ko reassign nahi kar sakte, lekin array ke contents push/splice se badal sakte ho.

forEach se new array expect karna
forEach returns undefined. New array banana ho to map() use karo.

10. Practice problems

1. Given [2, 4, 6], map() se har number ka square banao.

2. Given [5, 12, 18, 3], filter() se 10 se bade numbers nikalo.

3. Given [10, 20, 30], reduce() se sum nikalo.

4. Given [4, 9, 2, 15], reduce() ya Math.max se maximum find karo.

5. Given ["a", "b", "a", "c", "a"], reduce() se frequency object banao.

6. Students array mein se marks >= 80 wale students find karo.

7. Ek module math.js banao jo add aur subtract export kare; app.js mein import karo.

Practice answers (try first, then check)

```
// 1
[2, 4, 6].map(n => n * n); // [4, 16, 36]

// 2
[5, 12, 18, 3].filter(n => n > 10); // [12, 18]

// 3
[10, 20, 30].reduce((sum, n) => sum + n, 0); // 60

// 4
[4, 9, 2, 15].reduce((max, n) => n > max ? n : max, -Infinity); // 15

// 5
["a", "b", "a", "c", "a"].reduce((counts, ch) => {
    counts[ch] = (counts[ch] || 0) + 1;
    return counts;
}, {});
```

11. Quick revision sheet

Concept	Yaad rakhne wali baat
Function	Reusable code block; call karne par execute hota hai.
Parameter / argument	Parameter = definition placeholder; argument = passed value.
return	Function se value bahar bhejta hai.
Arrow function	Short function syntax: (x) => x * 2
Callback	Function jo doosre function ko argument mein pass hota hai.
HOF	Function jo function leta hai ya function return karta hai.
Array	Ordered collection; index 0 se shuru.
map	Transform each item; new array.
filter	Condition pass karne wale items; new array.
reduce	Accumulator se values combine; final result.
forEach	Har item par action; return undefined.
find	First matching element.
some / every	At least one / all items pass condition?
slice / splice	slice copies; splice changes original.
Modules	Files ke beech export/import se code share.

Best learning sequence: Functions → Parameters/return → Arrow functions → Arrays → Loops → map/filter/reduce → Objects in arrays → Modules → Mini projects.

End of guide. Keep practising: predict the output first, then run the code and explain why it happened.